FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 6

Journal de suivi, recherche et export

Durée : 5 jours Relations Query scopes Pagination Index MySQL Export CSV

Objectifs de la semaine

- tracer l'historique d'un incident (relation `hasMany`) ;
- écrire des **requêtes réutilisables** avec les *query scopes* ;
- paginer une liste en gardant les filtres ;
- comprendre comment MySQL utilise un **index** (`EXPLAIN`) ;
- produire un fichier **CSV** téléchargeable.

1. Le journal de suivi

```
// Modèle Incident
public function suivis(): HasMany
{
    return $this->hasMany(Suivi::class)->latest();   // du plus récent au plus ancien
}
```

Créer une entrée à partir de l'incident remplit `incident_id` tout seul :

```
$incident->suivis()->create([
    'user_id' => $request->user()->id,
    'type' => TypeSuivi::ChangementStatut,
    'message' => "Statut : {$ancien->label()} → {$incident->statut->label()}",
]);
```

Pour détecter un changement de statut, on mémorise l'ancienne valeur **avant** la mise à jour.

2. Les query scopes

Un scope est un filtre réutilisable, déclaré dans le modèle :

```
public function scopeActifs(Builder $query): void
{
    $query->whereIn('statut', [StatutIncident::Ouvert, StatutIncident::EnCours]);
}
```

Utilisation : `Incident::actifs()->count()`.

Pour des filtres optionnels, `when()` n'ajoute la condition que si la valeur est renseignée :

```
public function scopeFiltrer(Builder $query, array $filtres): void
{
    $query
        ->when($filtres['statut'] ?? null, fn ($q, $statut) => $q->where('statut', $statut))
        ->when($filtres['site_id'] ?? null, fn ($q, $site) => $q->where('site_id', $site))
        ->when($filtres['q'] ?? null, fn ($q, $texte) => $q->where(
            fn ($q) => $q->where('reference', 'like', "%{$texte}%")->orWhere('titre', 'like', "%{$texte}%")
        ));
}
```

Attention aux parenthèses — Sans la fonction qui entoure le `orWhere`, MySQL lirait `statut = 'ouvert' AND reference LIKE … OR titre LIKE …` et renverrait des incidents d'autres statuts. Vérifie avec `->toRawSql()`.

Le même scope sert à la liste ET à l'export : on écrit le filtrage une seule fois (principe DRY, *Don't Repeat Yourself*).

3. La pagination

```
$incidents = Incident::with(['site', 'assigneA'])
    ->filtrer($filtres)
    ->latest('debut_at')
    ->paginate(15)
    ->withQueryString();     // garde ?statut=ouvert dans les liens de page
```

Dans la vue : `{{ $incidents->links() }}`, et dans `AppServiceProvider` : `Paginator::useBootstrapFive();`.

4. MySQL : comment un index accélère une requête

```
EXPLAIN SELECT * FROM incidents WHERE statut = 'ouvert';
```

Regarde les colonnes `type`, `key` et `rows` :

- `type = ALL`, `key = NULL` → MySQL lit **toute** la table ;
- `type = ref`, `key = incidents_statut_index` → MySQL utilise l'index et ne lit que les lignes utiles.

À l'inverse, `LIKE '%texte%'` (avec un `%` au début) ne peut **pas** utiliser un index classique : c'est acceptable pour quelques milliers de lignes, pas pour des millions.

5. L'export CSV

```
return response()->streamDownload(function () use ($incidents) {
    $sortie = fopen('php://output', 'w');
    fwrite($sortie, "\xEF\xBB\xBF");                        // BOM : Excel comprend l'UTF-8
    fputcsv($sortie, ['Référence', 'Titre', 'Statut'], ';'); // ; = séparateur Excel français
    foreach ($incidents as $incident) {
        fputcsv($sortie, [$incident->reference, $incident->titre, $incident->statut->label()],
';');
    }
    fclose($sortie);
}, 'incidents.csv');
```

`streamDownload` envoie le fichier au fur et à mesure, sans le stocker sur le disque.

Exercices MySQL

1. Lancer `EXPLAIN` sur une requête filtrée par `statut`, puis sur une requête filtrée par `categorie`. Quelle différence ? Pourquoi ?
2. Rechercher les incidents dont le titre contient « fibre ».
3. Pour chaque incident, afficher le nombre de suivis (`LEFT JOIN suivis … GROUP BY`).
4. Afficher les 5 derniers suivis toutes incidences confondues, avec la référence de l'incident et le nom de l'auteur.
5. Bonus : `SELECT … LIMIT 15 OFFSET 30` correspond à quelle page ?

Travail demandé

1. **Journal de suivi** sur la page de détail :
   - formulaire type (Commentaire / Intervention terrain) + message ;
   - affichage en timeline : icône, type, auteur, date, message ;
   - entrées automatiques à la création (« Incident déclaré ») et à chaque changement de statut.
2. **Liste des incidents** :
   - colonnes : référence, titre + catégorie, sévérité, statut, site, assigné, début, durée ;
   - filtres : recherche, statut, sévérité, site, période (du / au), bouton réinitialiser ;
   - pagination par 15 qui conserve les filtres.
3. **Export CSV** des incidents filtrés, lisible dans Excel avec les accents.

Livrable

Le journal de suivi fonctionne ; la liste se filtre, se pagine et s'exporte.

Auto-évaluation

- [ ] Un changement de statut crée bien une entrée automatique dans le journal

- [ ] Le type « Changement de statut » ne peut pas être envoyé à la main par le formulaire
- [ ] Les filtres se combinent et restent actifs en changeant de page
- [ ] La liste et l'export utilisent le même scope
- [ ] Le CSV s'ouvre dans Excel sans caractères cassés
- [ ] Je sais lire le résultat d'un `EXPLAIN`

Ressources

- Laravel — Query scopes : laravel.com/docs/eloquent#local-scopes
- Laravel — Pagination : laravel.com/docs/pagination
- Laravel — Réponses de téléchargement : laravel.com/docs/responses#file-downloads
- MySQL — EXPLAIN : dev.mysql.com/doc/refman/8.4/en/explain-output.html

Pour aller plus loin

Envoyer un e-mail (Mailable, driver `log`) au technicien quand un incident lui est assigné.